NAME: TANKISO MASOEBE

PROGRAMMING WITH C++2 WK12

1. Implementing a Doubly Linked List

1. Introduction

A Doubly Linked List is a data structure where each node contains three parts:

- Data: Stores the value.
- Next: Points to the next node.
- Prev: Points to the previous node.

It allows us to move through the list in both forward and backward directions.

2. Python Implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None


class DoublyLinkedList:
    def __init__(self):
```

```
        self.head = None
        self.tail = None

    # Add a node at the end
    def append(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
        else:
            new_node.prev = self.tail
            self.tail.next = new_node
            self.tail = new_node

    # Add a node at the beginning
    def prepend(self, data):
        new_node = Node(data)
```

```
    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

# Display from beginning to end
def displayForward(self):
    current = self.head

    while current:
        print(current.data, end=" <-> ")
        current = current.next

    print("None")
```

```
    # Display from end to beginning
    def displayBackward(self):
        current = self.tail

        while current:
            print(current.data, end=" <-> ")
            current = current.prev

        print("None")


# Testing the doubly linked list
dll = DoublyLinkedList()

dll.append(10)
dll.append(20)
dll.append(30)
```

dll.prepend(5)

print("Forward:")

dll.displayForward()

print("Backward:")

dll.displayBackward()3. Output

Forward:
5 <-> 10 <-> 20 <-> 30 <-> None

Backward:
30 <-> 20 <-> 10 <-> 5 <-> None

4. Brief Report: Advantages and Disadvantages

Advantages

1. Reverse traversal: The list can be traversed forward and backward using `next` and `prev`.
2. Easy insertion: Adding a node at the beginning or end is efficient when the head and tail are maintained.
3. Easy deletion: A node can be removed more easily because it has a pointer to the previous node.
4. Flexible navigation: It is useful in browser history, music playlists, and undo/redo operations.

Disadvantages

1. More memory: Each node stores an extra `prev` pointer.
2. More complex: Both `next` and `prev` pointers must be updated correctly.
3. More operations: Insertion and deletion require updating more pointers, which can cause errors if handled incorrectly.

5. Comparison with a Singly Linked List

Feature	Singly Linked List	Doubly Linked List
Pointers per node	One (next)	Two (next, prev)
Forward traversal	Yes	Yes
Backward traversal	No	Yes
Memory usage	Lower	Higher
Insertion at beginning	\(O(1)\)	\(O(1)\)
Append with a tail pointer	\(O(1)\)	\(O(1)\)

6. Conclusion

A doubly linked list is more flexible than a singly linked list because it supports traversal in both directions. However, it uses more memory and requires more pointer management. It is useful when applications need to move forward and backward through data efficiently.

2. Doubly Linked List Implementation and Report

1. Introduction

A Doubly Linked List is a data structure where each node contains data, a pointer to the next node (next), and a pointer to the previous node (prev). It allows movement in both forward and backward directions.

2. Python Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
        self.prev = None



class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def append(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
        else:
            new_node.prev = self.tail
            self.tail.next = new_node
            self.tail = new_node
```

```
def prepend(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        self.tail = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

def displayForward(self):
    current = self.head

    while current:
        print(current.data, end=" <-> ")
        current = current.next
```

```
        print("None")

    def displayBackward(self):
        current = self.tail

        while current:
            print(current.data, end=" <-> ")
            current = current.prev

        print("None")


# Testing the doubly linked list
dll = DoublyLinkedList()

dll.append(10)
dll.append(20)
```

dll.append(30)

dll.prepend(5)

print("Forward:")

dll.displayForward()

print("Backward:")

dll.displayBackward()3. Output

Forward:
5 <-> 10 <-> 20 <-> 30 <-> None

Backward:
30 <-> 20 <-> 10 <-> 5 <-> None

4. Brief Report

A doubly linked list is useful because it allows traversal in both directions.

Advantages:

1. It supports easy forward and backward traversal.
2. Insertion at the beginning or end is efficient when the head and tail pointers are maintained.
3. Deleting a known node is easier because each node points to its previous node.
4. It is useful for browser history, music playlists, and undo/redo operations.

Disadvantages:

1. It uses more memory because each node stores an extra pointer.
2. Insertion and deletion require updating more pointers.
3. The implementation is more complex than a singly linked list.

5. Comparison

Feature	Singly Linked List	Doubly Linked List
Pointers	One	Two
Forward traversal	Yes	Yes
Backward traversal	No	Yes
Memory usage	Lower	Higher
Append with a tail pointer	\(O(1)\)	\(O(1)\)
Prepend	\(O(1)\)	\(O(1)\)

6. Conclusion

A doubly linked list provides more flexibility than a singly linked list because it supports movement in both directions. However, it requires more memory and careful pointer management. It is suitable for applications that need forward and backward navigation.